Dog Breed Classification Website

เว็บไซต์จำแนกสายพันธุ์ของสุนัข

Git Lab URL : https://gitlab.com/mtgu/Mairu.git (branch: veb)

จัดทำโดย

นายชคัตบดี ชัยถาวรกิจ 65070043

นายณัฐภัทร ทองมีขวัญ 65070080

นายรามิล ธรรมจินดา 65070198

นายอภิสิทธิ์ ปิดตาทะนัง 65070250

เสนอ

ดร.ทัศนัย พลอยสุวรรณ

โครงงานนี้เป็นส่วนหนึ่งของรายวิชา การดำเนินงานการเรียนรู้ของเครื่อง (06026241)

สาขาวิทยาการข้อมูลและการวิเคราะห์เชิงธุรกิจ คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ภาคเรียนที่ 2 ปีการศึกษา 2567

Problem Statement

ที่มาและความสำคัญ

ปัจจุบันการเลี้ยงสัตว์เป็นลูกเสมือนคนในครอบครัว หรือ “pet parent” กำลังได้รับความนิยมเพิ่มมากขึ้น แทนที่การเลี้ยงสัตว์แบบ pet lover ที่แค่อยากเลี้ยงเพราะรัก หรืออยากมีแค่สัตว์เลี้ยงเท่านั้น เมื่อค่านิยมของคนรุ่นใหม่เริ่มเปลี่ยนไป ประชากรในวัยทำงานมีแนวโน้มอยู่เป็นโสดและไม่แต่งงานกันมากขึ้น หรือแต่งงานแล้วไม่มีบุตร เมื่อเป็นผู้สูงอายุจึงมีแนวโน้มที่จะอยู่ตามลำพังคนเดียวหรือสองคนต่อไป การอยู่ตามลำพังทำให้เกิดความรู้สึกเหงา โดดเดี่ยว หรืออาจทำให้เกิดปัญหาสุขภาพทางจิตใจ เช่น ความซึมเศร้า ความสิ้นหวัง ความรู้สึกไร้ค่า สถานการณ์ดังกล่าวได้ทำให้ผู้คนพยายามหาที่พึ่งพิงทางจิตใจ เพื่อมาช่วยบำบัดจิตใจ คลายความเหงา หรืออยู่เป็นเพื่อน จึงทำให้สัตว์เลี้ยงได้เข้ามาเติมเต็ม และกลายเป็นส่วนหนึ่งของครอบครัวโดยไม่รู้ตัว การปฏิสัมพันธ์ระหว่างมนุษย์กับสัตว์ (Human-animal interactions: HAI) จึงกลายเป็นส่วนสำคัญในชีวิตของผู้คนจำนวนมากในทุกวัย และสัตว์เลี้ยงกลายเป็นส่วนหนึ่งของสมาชิกในครอบครัว

การสำรวจประชากรสุนัขและแมวในปี 2566 โดยศูนย์บัญชาการเพื่อการเฝ้าระวังโรคพิษสุนัขบ้า คณะสัตวแพทยศาสตร์ มหาวิทยาลัยเกษตรศาสตร์1 พบว่า จำนวนประชากรสุนัขและแมวที่มีเจ้าของ มีจำนวนเพิ่มขึ้นอย่างมากในทุกปี โดยในปี 2566 สุนัขและแมวที่มีเจ้าของมีจำนวนสูงถึง 5.8 ล้านตัว เป็นสุนัข 3.8 ล้านตัว และแมว 2.0 ล้านตัว การมีสัตว์เลี้ยงเข้ามาเป็นส่วนหนึ่งในครอบครัว เป็นปรากฏการณ์สวนกระแสกับจำนวนเด็กเกิดของประเทศไทยในปัจจุบัน ซึ่งในปี 2566 สำนักบริหารการทะเบียน กระทรวงมหาดไทย รายงานว่า มีจำนวนเด็กเกิดใหม่เพียง 517,934 คนเท่านั้น

นอกจากนี้ ปัญหาเรื่องสุนัขจรจัดก็เป็นปัญหาเช่นเดียวกัน ที่ส่งผลกระทบต่อสวัสดิภาพของสัตว์ คนในสังคม สาธารณะสุข สังคมและสิ่งแวดล้อม

จากปัญหาข้างต้น เราจึงได้ทำเว็บไซต์ทำนายสายพันธุ์สุนัขตอบสนองปัญหาดังกล่าว โดยการใช้ Machine Learning เข้ามาใช้ในการทำนาย และเนื่องจากความก้าวหน้าของ Deep Learning และ Computer Vision ที่มีความก้าวหน้าอย่างมาก ทำให้สามารถพัฒนาโมเดลจำแนกสายพันธุ์สุนัขจากภาพถ่ายได้อย่างแม่นยำ ระบบเหล่านี้สามารถช่วยให้เจ้าของสุนัขได้รับข้อมูลเกี่ยวกับสายพันธุ์ของสุนัขของพวกเขา ภายในเวลาไม่กี่วินาที โดยไม่จำเป็นต้องพึ่งสัตวแพทย์หรือผู้เชี่ยวชาญด้านสัตว์เลี้ยง

นอกจากนี้ยังมี ปัญหาด้านการดูแลสุนัขอย่างถูกต้อง เนื่องจาก สุนัขแต่ละสายพันธุ์มีความต้องการที่แตกต่างกัน เช่น สุนัขบางสายพันธุ์ต้องการ ออกกำลังกายมาก (เช่น บอร์เดอร์คอลลี่) ในขณะที่บางพันธุ์เหมาะสำหรับการอยู่ในบ้าน (เช่น บูลด็อก) หรือสุนัขบางพันธุ์มี แนวโน้มเป็นโรคทางพันธุกรรมสูง (เช่น ลาบราดอร์มีแนวโน้มเป็นโรคข้อสะโพกเสื่อม) และอื่นๆ

โภชนาการของสุนัขแต่ละพันธุ์แตกต่างกัน เช่น สุนัขบางพันธุ์ต้องการอาหารที่มีโปรตีนสูงกว่าพันธุ์อื่น ดังนั้น ระบบจำแนกสายพันธุ์สุนัขอัตโนมัติ ที่สามารถให้ข้อมูลสำคัญเกี่ยวกับ ลักษณะนิสัย อาหารที่เหมาะสม ปัญหาสุขภาพ และการดูแลที่ถูกต้อง จะเป็น เครื่องมือที่ช่วยให้เจ้าของสามารถเลี้ยงดูสุนัขได้อย่างเหมาะสมยิ่งขึ้น

Solution

เพื่อแก้ไขปัญหาที่กล่าวมา Dog Breeds Classification Website จะถูกออกแบบมาให้ทำงานแบบ AI-driven Web Application โดยใช้ Machine Learning และ Computer Vision เพื่อจำแนกสายพันธุ์สุนัขจากภาพถ่าย พร้อมทั้งให้คำแนะนำด้านโภชนาการ การดูแล และสุขภาพที่เหมาะสม

Target

- กลุ่มเจ้าของสุนัข
- กลุ่มคนรักสุนัข
- กลุ่มผู้รับเลี้ยงสุนัข
- กลุ่มคนที่สนใจในการเลี้ยงสุนัข

แนวทางการแก้ไขปัญหา (Solution Approach)

1. ใช้ Machine Learning Model สำหรับการจำแนกสายพันธุ์สุนัข
   - ใช้ Deep Learning (CNN – Convolutional Neural Network) เช่น ResNet, MobileNet, EfficientNet สำหรับ Image Classification
   - พัฒนา Custom Model หรือ Fine-tuning Pretrained Model บนชุดข้อมูลสายพันธุ์สุนัข
   - ใช้ Transfer Learning เพื่อลดการใช้ทรัพยากรในการฝึกโมเดลและเพิ่มความแม่นยำ

2. ระบบรับภาพและประมวลผลอัตโนมัติ

- ผู้ใช้สามารถ อัปโหลดภาพผ่านเว็บไซต์ หรือ กล้องมือถือ
- ใช้ Image Preprocessing เพื่อลดสัญญาณรบกวน เช่น Resizing, Normalization, Data Augmentation
- ส่งภาพเข้าสู่ AI Model เพื่อคาดการณ์สายพันธุ์

3. แสดงผลข้อมูลเกี่ยวกับสายพันธุ์สุนัข

- หลังจากโมเดลทำนายสายพันธุ์แล้ว ระบบจะแสดงข้อมูลต่อไปนี้
- ชื่อสายพันธุ์ พร้อมรูปภาพประกอบ
- น้ำหนักเฉลี่ย, ส่วนสูง, อายุขัยของสุนัข
- ลักษณะนิสัยของสุนัขแต่ละพันธุ์
- สุขภาพ และโรคที่พบบ่อย
- อาหารที่แนะนำสำหรับแต่ละสายพันธุ์
- คำแนะนำด้านการออกกำลังกาย และการดูแลทั่วไป

4. การเก็บข้อมูล Feedback และ Retraining Model

- ให้ ผู้ใช้สามารถให้ Feedback ว่าผลลัพธ์ที่โมเดลพยากรณ์ถูกต้องหรือไม่
- ถ้าผู้ใช้แก้ไขสายพันธุ์ ระบบจะ เก็บข้อมูลในฐานข้อมูล และนำไปใช้ Retrain โมเดล เพื่อเพิ่มความแม่นยำ และทำ active learning เพื่อให้โมเดลสามารถพัฒนาจากข้อมูลใหม่ๆ

5. ใช้ Cloud Storage และ MLOps เพื่อจัดการข้อมูลโมเดล

- ใช้ Google Cloud Storage (GCS) Dataset & Model Artifacts
- ใช้ DVC (Data Version Control) ควบคุมเวอร์ชันของข้อมูล Training Data & Model Checkpoints
- ใช้ CI/CD Pipeline (เช่น GitHub Actions, Google Cloud Build) เพื่อติดตั้งโมเดลใหม่โดยอัตโนมัติ

Sustainability รายได้

- ให้ธุรกิจที่ต้องการใช้ AI จำแนกสุนัข (เช่น ศูนย์พักพิง, องค์กรสัตวแพทย์) เช่า API ของระบบ
- Partnership & Sponsorship
    - ร่วมมือกับแบรนด์ที่เกี่ยวข้องในการลงโฆษณา หรือเป็นผู้สนับสนุนให้กับแพลตฟอร์ม

Sustainability unfair advantage

- ระบบที่ทำการเทรนโมเดลใหม่ อัตโนมัติทุกๆ 3 วัน
- ทำให้โมเดลในการทำนายมีประสิทธิภาพที่ดีมากยิ่งขึ้น

Social Impact

การพัฒนา Dog Breed Classification Website ไม่เพียงแต่ช่วยอำนวยความสะดวกให้กับเจ้าของสุนัข แต่ยังส่งผลกระทบต่อสังคมในหลายด้าน ตั้งแต่การพัฒนาคุณภาพชีวิตของสัตว์เลี้ยง การส่งเสริมความรับผิดชอบของเจ้าของ ไปจนถึงการสนับสนุนองค์กรช่วยเหลือสัตว์ ได้แก่

1. เพิ่มคุณภาพชีวิตของสุนัขและเจ้าของ

   ระบบช่วยให้เจ้าของ เข้าใจสายพันธุ์ของสุนัขอย่างถูกต้อง ทำให้สามารถดูแลสุนัขให้เหมาะสมกับสายพันธุ์ เช่น

   - การเลือก อาหาร ที่เหมาะสมกับพันธุ์สุนัข
   - การออกกำลังกายและกิจกรรมที่เหมาะสม
   - การเฝ้าระวังโรคที่มักพบในแต่ละสายพันธุ์
   - ลดปัญหาการดูแลผิดวิธีที่อาจทำให้สุนัขมีปัญหาด้านสุขภาพ
2. ลดปัญหาการถูกทอดทิ้งและการเลือกซื้อสุนัขอย่างไม่รับผิดชอบ

   เจ้าของบางคนอาจซื้อสุนัขโดยไม่ได้ศึกษาลักษณะนิสัยของสายพันธุ์ ส่งผลให้เกิดปัญหาด้านพฤติกรรม เช่น

   - พฤติกรรมก้าวร้าวของสุนัขบางสายพันธุ์ที่ต้องการการฝึกอย่างเข้มงวด
   - สุนัขที่ต้องการพื้นที่กว้างขวางแต่ถูกเลี้ยงในพื้นที่จำกัด
   - เมื่อเจ้าของเข้าใจนิสัยของสายพันธุ์ตั้งแต่ต้น จะช่วยลดอัตราการนำสุนัขไปทิ้ง เพราะเลี้ยงไม่ไหว
3. สนับสนุนการรับเลี้ยงสุนัข (Adoption) แทนการซื้อ
   - สำหรับสุนัขที่ไม่มีเอกสารรับรองสายพันธุ์ เช่น สุนัขในศูนย์พักพิง ระบบสามารถช่วยเจ้าของใหม่ คาดการณ์สายพันธุ์ของสุนัขได้

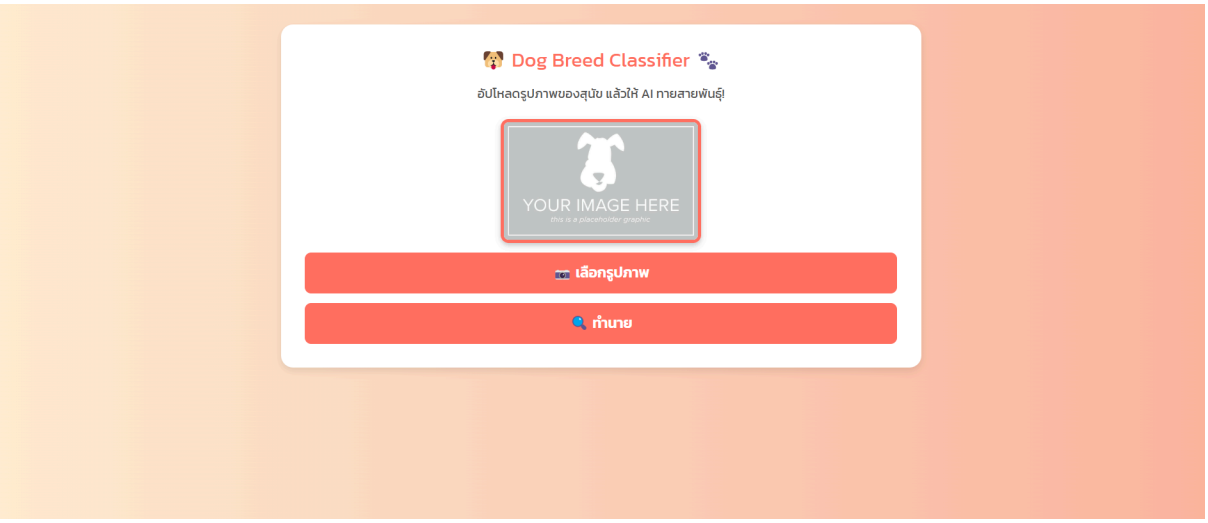
- ทำให้เจ้าของสามารถตัดสินใจรับเลี้ยงได้ง่ายขึ้น โดยรู้ว่าสุนัขสายพันธุ์นั้นๆ มีนิสัยและความต้องการอย่างไร

4. สนับสนุนการศึกษาและการเรียนรู้เกี่ยวกับสัตว์เลี้ยง
    - เว็บไซต์สามารถเป็นแหล่งข้อมูล สำหรับเจ้าของมือใหม่ ที่ต้องการศึกษาการเลี้ยงดูสุนัขแต่ละสายพันธุ์
    - องค์กรที่เกี่ยวข้องกับสัตว์สามารถ นำข้อมูลจากระบบไปใช้ให้ความรู้แก่สาธารณะ เช่น โรงเรียนสัตวแพทย์ หรือองค์กรช่วยเหลือสัตว์

Website

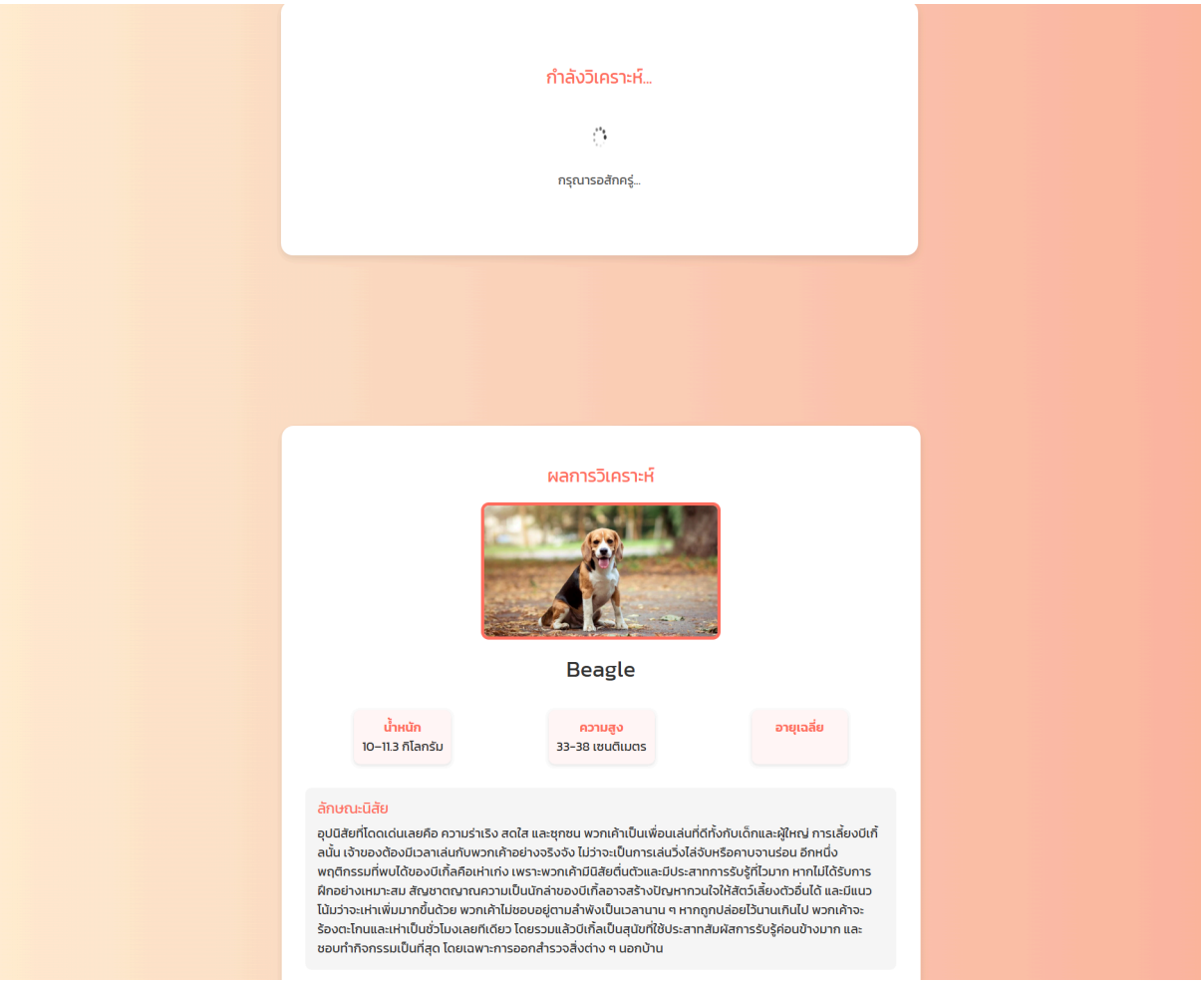
Web pages

1. หน้าหลัก (Home Page)

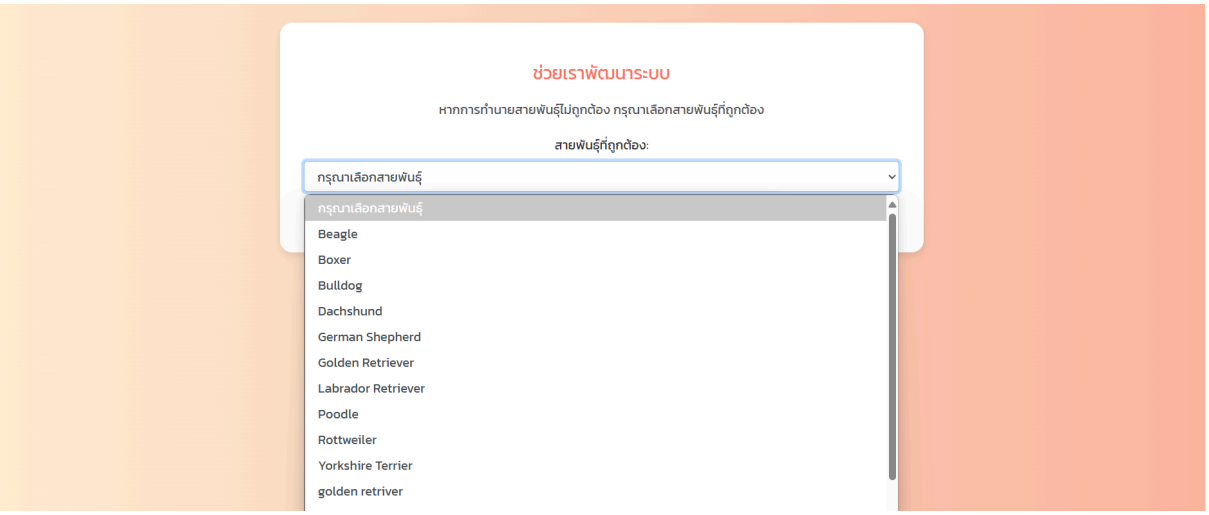


- User อัพโหลดรูปสุนัขที่ต้องการจะให้เว็บไซต์จำแนกสายพันธุ์

2. หน้าประมวลผล



4. หน้าส่ง feedback



หลักการทำงาน Frontend

- **เทคโนโลยีที่ใช้**
    - HTML, CSS, JavaScript (UI และการจัดการ DOM)
    - Bootstrap (ปรับแต่ง UI)
    - Fetch API (AJAX) (ใช้สื่อสารกับ Backend)
    - Flask Jinja2 Template (ใช้ดึงข้อมูลจาก Backend มาแสดงผล)
- **หน้าที่ของ Frontend**
    1. อนุญาตให้ผู้ใช้อัปโหลดรูปภาพสุนัข
    2. ส่งรูปไปที่ API /api/predict
    3. รอผลการพยากรณ์จาก AI
    4. แสดงผลลัพธ์ที่ได้รับจาก API เช่น ชื่อสายพันธุ์ น้ำหนัก ส่วนสูง
    5. ให้ผู้ใช้สามารถส่งฟีดแบ็ค หากผลลัพธ์ไม่ถูกต้อง
- **กระบวนการทำงานของ Frontend**
    1. อัปโหลดรูปภาพ

```html
<form id="upload-form" enctype="multipart/form-data">
  <div class="upload-container">
    <div id="preview-container" class="hidden">
      <img id="preview-image" src="" alt="Preview">
    </div>
    <img id="placeholder-image" src="{{ url_for('static', filename='images/placeholder-dog.png') }}" alt="Upload Preview">
  </div>
  <input type="file" id="image-input" name="image" accept=".png, .jpg, .jpeg" required style="display: none;">
  <label for="image-input" class="custom-file-upload">📷 เลือกรูปภาพ</label>
  <button class="upload-btn" type="submit">🔍 ทำนาย</button>
```

        - ผู้ใช้กดปุ่ม "เลือกรูปภาพ"
        - แสดงตัวอย่างรูปภาพที่อัปโหลด (preview-image)
    2. ส่งรูปไปที่ API
        - เมื่อผู้ใช้กด "ทำนาย"
        - JavaScript ใช้ Fetch API ส่งรูปไปยัง API /api/predict

```javascript
// ส่ง API request
fetch('/api/predict', {
    method: 'POST',
    body: formData
})
```

        - ตัวอย่างการใช้ fetch api ใน app.js

3. แสดงผลลัพธ์

- เมื่อ API ตอบกลับมา
- JavaScript จะนำข้อมูลไปแสดงที่ <section id="result-section">
- ใช้ Template (dog-detail-template) ในการแสดงผล

```javascript
// แสดงผลลัพธ์การวิเคราะห์
function displayDogBreedResult(dogData) {
    // คัดลอก template
    const template = document.getElementById('dog-detail-template');
    const content = template.innerHTML;

    // แสดงในส่วนผลลัพธ์
    resultSection.innerHTML = content;
    resultSection.classlist.remove('hidden');

    // เติมข้อมูล
    document.getElementById('dog-img').src = '/static' + dogData.pic;
    document.getElementById('dog-name').textContent = dogData.name;
    document.getElementById('dog-weight').textContent = dogData.weight;
    document.getElementById('dog-height').textContent = dogData.height;
    document.getElementById('dog-avg-age').textContent = dogData.avg_age;
    document.getElementById('dog-character').innerHTML = dogData.character;
    document.getElementById('dog-health').innerHTML = dogData.health;
    document.getElementById('dog-food').innerHTML = dogData.food;
    document.getElementById('dog-care').innerHTML = dogData.care;
```

4. ให้ฟีดแบ็คหากผลลัพธ์ไม่ถูกต้อง

- มีปุ่ม "การทายสายพันธุ์ผิด โปรดช่วยเราพัฒนา"
- เมื่อกดปุ่ม จะมี Dropdown ให้เลือกสายพันธุ์ที่ถูกต้อง
- ซึ่งหากไม่มีให้เลือก ผู้ใช้จะสามารถเพิ่มสายพันธุ์ได้

```html
<!-- Feedback Section -->
<section id="feedback-section" class="hidden">
  <h2>ช่วยเราพัฒนาระบบ</h2>
  <p>หากการทำนายสายพันธุ์ไม่ถูกต้อง กรุณาเลือกสายพันธุ์ที่ถูกต้อง</p>

  <form id="feedback-form">
    <div class="form-group">
      <label for="breed-dropdown">สายพันธุ์ที่ถูกต้อง:</label>
      <select id="breed-dropdown" name="breed_dropdown" class="form-control" required>
        <option value="" disabled selected>กรุณาเลือกสายพันธุ์</option>
        <!-- Options will be populated by JavaScript -->
      </select>

      <div id="other-breed-container" class="hidden mt-3">
        <label for="other-breed">ระบุสายพันธุ์อื่นๆ:</label>
        <input type="text" id="other-breed" class="form-control" placeholder="ระบุสายพันธุ์">
      </div>

      <input type="hidden" id="feedback-filename" name="filename">
    </div>
    <div class="buttons-group">
      <button type="submit" class="submit-feedback-btn">ส่งข้อมูล</button>
      <button type="button" id="cancel-feedback-btn" class="cancel-btn">ยกเลิก</button>
    </div>
  </form>
```

- เมื่อกด "ส่งข้อมูล" (submit-feedback-btn)
- JavaScript ใช้ Fetch API ส่งข้อมูลไปที่ /api/feedback เพื่อบันทึกลงฐานข้อมูล

```javascript
fetch('/api/feedback', {
    method: 'POST',
    body: formData
})
```

หลักการทำงาน Backend

- **เทคโนโลยีที่ใช้**
    - Flask (Python) (Web Framework)
    - Machine Learning Model (TensorFlow, Scikit-learn, MLflow) (พยากรณ์สายพันธุ์)
    - SQLite Database (เก็บข้อมูลสายพันธุ์)
    - Google Cloud Storage (GCS) (จัดเก็บรูปภาพ)
    - DVC (Data Version Control) + Git (ใช้จัดการเวอร์ชั่นข้อมูล)
- **หน้าที่ของ Backend**
    1. รับภาพจากผู้ใช้และส่งไปให้โมเดลพยากรณ์
    2. ดึงข้อมูลสายพันธุ์จากฐานข้อมูล
    3. จัดการฟีดแบ็คที่ได้รับและบันทึกลงฐานข้อมูล
    4. รองรับ API ที่ใช้โดย Frontend
- **หลักการทำงานของ Backend**
    1. **การรับภาพและทำนายผล**
        a. โหลดโมเดลสำหรับจะใช้ประมวลผล

```python
def get_dog_classifier():
    """Load the dog classification model from file"""
    try:
        if not os.path.exists(MODEL_PATH):
            logger.error(f"Model file not found: {MODEL_PATH}")
            raise FileNotFoundError(f"Model file not found: {MODEL_PATH}")

        logger.info(f"Loading model from: {MODEL_PATH}")
        with open(MODEL_PATH, 'rb') as file:
            model = pickle.load(file)
        logger.info("Model loaded successfully")
```

- pickle ในการโหลดโมเดล .pkl

b. รับภาพจากผู้ใช้ผ่าน API /api/predict

- Flask รับภาพจาก request.files['image']
- ใช้ Pillow (PIL) แปลงภาพเป็นรูปแบบที่โมเดลต้องการ

- โหลดโมเดล (dogmodel.pkl) และพยากรณ์สายพันธุ์

```python
def model_predict(img, model):
    """แปลงภาพให้เข้ากับโมเดล และทำการพยากรณ์"""
    x = img.resize((150, 150))
    x = np.array(x) / 255.0  # Normalization
    pred = model.predict(x.reshape(1, 150, 150, 3))
    return pred
```

- ตัวอย่างใน dogapp.py

c. ดึงข้อมูลจากฐานข้อมูล

- เมื่อพยากรณ์สายพันธุ์ได้แล้ว
- ใช้ SQLite Database ดึงรายละเอียดของสายพันธุ์

```python
# ค้นหาข้อมูลของสุนัขในฐานข้อมูล
db = get_db()
cursor = db.cursor()
cursor.execute("SELECT * FROM DogBreed WHERE name = ?", (result,))
query_result = cursor.fetchone()
```

d. ส่ง JSON Response กลับไปให้ Frontend

2. การให้ feedback

a. ดึงข้อมูลสายพันธุ์ต่างๆ จากโฟลเดอร์ใน temp data แล้วนำมาทำเป็น list

```python
def api_breeds():
    """API endpoint to return all available dog breeds from GCS bucket"""
    try:
        # Use the get_breeds_from_gcs function to fetch breed names from GCS
        all_breeds = get_breeds_from_gcs()

        # Include the labels from the LABEL_DICT as well
        for breed in LABEL_DICT.values():
            if breed not in all_breeds:
                all_breeds.append(breed)

        # Sort the breeds list
        all_breeds = sorted(all_breeds)

        current_app.logger.info(f"Retrieved {len(all_breeds)} breeds: {all_breeds}")

        return jsonify({
            "success": True,
            "breeds": all_breeds
```

b. เมื่อผู้ใช้ต้องการให้ฟีดแบ็ค (API /api/feedback)

- รับ Session ID จาก Cookie
- รับ สายพันธุ์ที่ถูกต้อง ที่ผู้ใช้เลือกจาก Dropdown
- อัปโหลดรูปที่ได้รับไปยัง Google Cloud Storage (GCS)
- หากเป็นสายพันธุ์ใหม่ อัปเดต label_dict

c. เรียกใช้งาน DVC เพื่อจัดเก็บเวอร์ชันข้อมูล

- หลังผ่านไปทุกๆ 3 วันของระบบ จะมีการใช้งานใช้งาน DVC เพื่อจัดเก็บเวอร์ชันข้อมูล โดยจะนำข้อมูลจาก temp data ไปทำเวอร์ชัน และเก็บลงไฟล์ถาวร (dog-data)
- ใช้ Git + DVC ติดตามเวอร์ชันของข้อมูล

3. การฝึกโมเดลอัตโนมัติ

a. ใช้ APScheduler ตั้งเวลาให้ Train Model ทุก 3 วัน

```
scheduler.add_job(
    scheduled_training_job,
    'interval',
    days=3,
    id='regular_training',
    replace_existing=True
)
```

b. กระบวนการ Train Model

- ดาวน์โหลดข้อมูลจาก dog-data
- โหลดโมเดลที่ใช้งานอยู่ และทำการ Fine-tuning
- บันทึกเวอร์ชันใหม่ของโมเดลใน MLflow
- อัปโหลดโมเดลใหม่ไปที่ bucket ที่มี folder ชื่อ models

Database

เป็นฐานข้อมูล SQLite ที่ใช้เก็บข้อมูลของสายพันธุ์สุนัข ซึ่งระบบ Flask Backend จะใช้เพื่อดึงรายละเอียดของสายพันธุ์ที่ทำนายได้ และแสดงผลในหน้าเว็บ

โดยจะประกอบไปด้วย 10 คอลัมน์ ได้แก่

1. Column id (Primary key) (int)

id ▲
Filter
1
2
3
4
5
6
7
8
9
10

2. Column pic (text)

   - แสดง path รูปภาพของสุนัขสำหรับแสดงบนหน้าเว็บ

pic
Filter
images\Golden4.jpg
images\german-shepherd-3.webp
images\labradorretriever2.jpeg
images\bulldog.webp
images\beagle-dog.avif
images\poodle.jpg
images\rotweiler.png
images\terrier.jpg
images\boxer.webp
images\dachshund.webp

3. Column name (text)

- แสดงชื่อของสายพันธุ์สุนัข

name
Filter
Golden Retriever
German Shepherd
Labrador Retriever
Bulldog
Beagle
Poodle
Rottweiler
Yorkshire Terrier
Boxer
Dachshund

4. Column weight (text)

- แสดงน้ำหนักเฉลี่ยของสายพันธุ์

weight
Filter
27-34 กิโลกรัม
22-40 กิโลกรัม
25-36 กิโลกรัม
18-30 กิโลกรัม
10-11.3 กิโลกรัม
20-30 กิโลกรัม
36-61 กิโลกรัม
1-3 กิโลกรัม
30-31 กิโลกรัม
4-14.5 กิโลกรัม

5. Column height (text)

- แสดงส่วนสูงเฉลี่ยของสายพันธุ์

height
Filter
51-61 เซนติเมตร
58-63 เซนติเมตร
55-57 เซนติเมตร
40 เซนติเมตรไปขึ้นไป
33-38 เซนติเมตร
38 เซนติเมตรขึ้นไป
56-69 เซนติเมตร
20-22 เซนติเมตร
58.5-63.5 เซนติเมตร
12-22 เซนติเมตร

6. Column avage (text)

- แสดงอายุเฉลี่ยของสายพันธุ์

avage
Filter
10-12 ปี
9-13 ปี
10-14 ปี
8-10 ปี
12-15 ปี
10-18 ปี
9-10 ปี
12-15 ปี
10-12 ปี
12-15 ปี

7. Column character (text)

- แสดงลักษณะนิสัย

character
Filter
เป็นสุนัขที่เป็นมิตรและนิสัยดีเป็นอันดับต้นๆ ในบรรดาสุนัขทั้งหมด แต่เดิมสุนัขโกลเด้น รีทรีฟเวอ...
เป็นที่สุนัขที่จะผูกพันกับผู้ดูแลและอยากใช้เวลาด้วยกันให้มากที่สุดเท่าที่จะเป็นไปได้ ...
ขึ้นชื่อในเรื่องของความฉลาดหลักแหลม เป็นมิตร และนิสัยดี จี้ประจบเอาใจทั้งคนและสัตว์เลี้ยงตัว...
สายพันธุ์นี้เป็นตัวอย่างของคำพูดที่ว่า "อย่าตัดสินหนังสือจากปก" ...
อุปนิสัยที่โดดเด่นเลอคือ ความร่าเริง สดใส และซุกซน พวกเค้าเป็นเพื่อนเล่นที่ดีทั้งกับเด็กและผู้ให...
หากต้องการเลี้ยงพุดเดิ้ล การทำความเข้าใจลักษณะเฉพาะตัวของพวกเค้าเป็นสิ่งสำคัญมาก ...
เป็นสุนัขที่เชื่อฟังมาก และสามารถฝึกได้ง่าย มีความจงรักภักดีสูง ทั้งรักและติดเจ้าของ ...
มีความแอคทีฟสูง ตื่นตัวตลอดเวลา และเต็มไปด้วยความอยากรู้อยากเห็น พวกเค้าอาจจะดื้อดึงไปบ้า...
มีความกระตือรือร้น ตื่นตัว และสนใจสิ่งรอบข้างเป็นอย่างมาก ...
แม้ว่าดัชชุนจะเป็นนักกิจกรรมด้วยง แต่ในขณะเดียวกันก็สามารถเลี้ยงในบ้านได้ และพวกเค้าเข้ากับเด็...

8. Column health (text)

- แสดงสุขภาพ

health
Filter
ถึงแม้ว่าสุนัขโกลเด้น รีทรีฟเวอร์ จะถูกจัดเป็นสุนัขพันธุ์ใหญ่ ...
เป็นสุนัขสายพันธุ์ที่มีปัญหาสุขภาพหลายประการ ได้แก่ โรคระบบทางเดินอาหา...
เสี่ยงเป็นโรคตาที่เกิดจากกรรมพันธุ์เหมือนกับฮีกหลายสายพันธุ์ ...
บริติช บูลด็อกเติบโตได้ดีในสภาพอากาศอบอุ่น อากาศร้อนทำให้หายใจลำบาก...
สุนัขปีเกี้ลไม่ค่อยมีปัญหาสุขภาพทางพันธุกรรม แต่เช่นเดียวกับสุนัขทุกสายพันธุ์...
สุนัขพันธุ์พุดเดิ้ลส่วนใหญ่มีสุขภาพดี แต่ก็อาจเจ็บป่วยได้เช่นกัน ...
โดยทั่วไปร็อตไวเลอร์จัดเป็นสุนัขที่มีสุขภาพดี ...
โรคกระดูกสะบ้าเคลื่อน โรคภูมิแพ้ ปัญหาเกี่ยวกับข้อต่อและสายตา ...
สุนัขพันธุ์บ็อกเซอร์ เป็นสุนัขที่สุขภาพแข็งแรง แต่สามารถเกิดการเจ็บป่วย ...
เนื่องจากลักษณะทางร่างกาย ...

9. Column food (text)

- แสดงอาหารที่เหมาะสม

food
Filter
สุนัขพันธุ์นี้มีแนวโน้มจะอ้วนได้ง่าย ถ้าไม่ควบคุมปริมาณอาหารให้ดี ...
สุนัขพันธุ์ใหญ่ มื้ออาหารก็ย่อมใหญ่ตามไปด้วย ...
สุนัขควรได้รับอาหารแห้งที่มีคุณภาพสูง 3–4 ถ้วยต่อวัน ...
พวกเค้าต้องการโปรตีนทั้งจากพืชและสัตว์ เพื่อเสริมสร้างกระดูก เส้นขน ...
ปีเกิ้ลเป็นจอมตะกละที่พร้อมจะกินทุกอย่างที่ขวางหน้า ...
ปริมาณอาหารที่แนะนำต่อวันสำหรับพุดเดิ้ล สแตนดาร์ดคืออาหารที่มีคุณภาพสูง...
สำหรับร็อดไวเลอร์อาหารเม็ดปริมาณ 2 – 5 ถ้วย ...
ควรกินอาหารหมาวันละ 2 มื้อ ...
ควรได้รับอาหารที่มีประโยชน์ ปริมาณอาหารที่แนะนำคือ 2 ถ้วยต่อวัน ...
สุนัขขนาดเล็กมีอัตราเผาผลาญพลังงานสูง ย่อยอาหารได้รวดเร็ว ...

10. Column care (text)

- แสดงการดูแล

care
Filter
เนื่องจากเป็นสุนัขที่พลังงานเยอะ บวกกับชอบเล่น ชอบทำกิจกรรมนอกบ้าน ...
ควรหมั่นพาลูกสุนัขเยอรมันเชพเพิร์ดไปออกกำลังกายอย่างสม่ำเสมอ ...
ริทริฟเวอร์เป็นสุนัขที่มีพลังงานเยอะ ...
ชอบนอนพอ ๆ กับชอบออกไปเดินเล่น แต่เนื่องจากมีรูปร่างเตี้ยล่ำสัน ...
ปีเกิ้ลมีความต้องการออกกำลังกายสูง เนื่องจากเป็นสุนัขที่พลังงานเยอะ ...
พุดเดิ้ลทุกขนาดมีความแอคทีฟสูง พวกเค้าจำเป็นต้องออกกำลังกายเป็นประจำ ...
ร็อดไวเลอร์เป็นสุนัขที่อการใช้งาน มีความขยันขันแข็ง ...
เป็นสายพันธุ์ที่แอคทีฟและชอบเล่นมาก ๆ พวกเค้าชอบเดินเล่นในสวน ...
เป็นสุนัขสุขภาพแข็งแรง มีพลังงานมาก ...
ดัชชุนมีนิสัยรักการกินและเชื่อง่าย ...

MLops

1. Data Pipeline

- ใช้ Google Cloud Storage (GCS) สำหรับจัดเก็บและเตรียมข้อมูลภาพของสุนัขแบบต่อเนื่อง
- ใช้ DVC (Data Version Control) ร่วมกับ Git เพื่อเก็บและจัดการ Data, Model และ Code Versioning
- ระบบสามารถ อัปโหลดและเก็บข้อมูล Feedback ของผู้ใช้งานได้แบบต่อเนื่อง

```python
def upload_to_gcs(file_bytes, label, filename, temp=True):
    """
    อัปโหลดไฟล์ไปยัง Google Cloud Storage

    Args:
        file_bytes: bytes of the image
        label: dog breed label
        filename: name of the file
        temp: if True, uploads to temporary directory, otherwise to dog-data
    """
    try:
        # Create a storage client
        storage_client = storage.Client()

        # Get the bucket
        bucket = storage_client.bucket(GCS_BUCKET_NAME)

        # Define blob path with label as folder name
        base_path = GCS_TEMP_DATA_PATH if temp else GCS_DOG_DATA_PATH
        blob_path = f"{base_path}/{label}/{filename}"
        blob = bucket.blob(blob_path)

        # Upload from memory
        blob.upload_from_string(file_bytes, content_type='image/jpeg')

        # Return the public URL
        return f"gs://{GCS_BUCKET_NAME}/{blob_path}"
    except Exception as e:
        app.logger.error(f"Error uploading to GCS: {str(e)}")
        return None
```

2. Model Training & Experiment Tracking

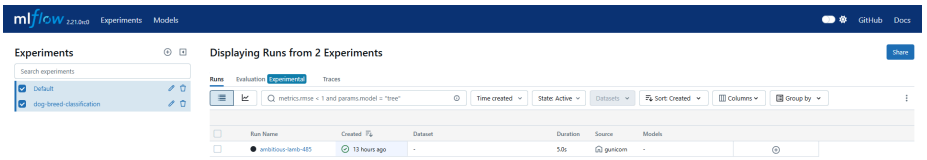
- ใช้ MLflow ในการติดตามการทดลองและบันทึก hyperparameters, metrics และ model artifacts
- มีการกำหนด MLflow Tracking URI สำหรับบันทึกผลลัพธ์ของโมเดลที่ฝึก

- สามารถทำ Retraining โมเดลใหม่ได้โดยอัตโนมัติเมื่อครบเวลาที่กำหนด

```python
def train_model():
    """
    Train model using data from dog-data directory with MLflow tracking
    """
    app.logger.info("Starting model training process...")

    try:
        # Configure MLflow
        mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
        mlflow.set_experiment(EXPERIMENT_NAME)

        # Download training data
        train_data_dir = os.path.join(TEMP_DIR, "training_data")
        download_training_data(train_data_dir)

        # Load current model for fine-tuning
        current_model = get_dog_classifier()

        # Start MLflow run
        with mlflow.start_run() as run:
            run_id = run.info.run_id
```

3. Continuous Integration / Continuous Deployment (CI/CD)

- สามารถทำ Retraining โมเดลใหม่ได้โดยอัตโนมัติเมื่อครบเวลาที่กำหนด
- จัดการข้อมูลที่เข้ามาอัตโนมัติ ให้อยู่ในรูปแบบที่ง่ายต่อการใช้งาน



-

```python
# Initialize scheduler if needed
if run_scheduler:
    @app.before_first_request
    def initialize_scheduler():
        """Initialize and start the scheduler for regular tasks"""
        if not scheduler.running:
            from app.training import scheduled_task

            # Schedule tasks every TRAINING_INTERVAL_DAYS days
            scheduler.add_job(
                scheduled_task,
                'interval',
                days=TRAINING_INTERVAL_DAYS,
                id='regular_tasks',
                replace_existing=True
            )
            scheduler.start()
            app.logger.info(f"Scheduler started. Will run every {TRAINING_INTERVAL_DAYS} days.")
else:
    app.logger.info("Scheduler disabled for this instance.")
```

```python
def scheduled_task():
    """
    Function to be scheduled for periodic DVC versioning and model training
    """
    logger.info("Running scheduled DVC versioning and training job...")

    # First run DVC version control
    version_id = run_dvc_for_data_versioning()

    if version_id:
        # Then train the model
        run_id = train_model()

        if run_id:
            logger.info(f"Scheduled tasks completed successfully. Version: {version_id}, Run ID: {run_id}")
            return True
        else:
            logger.error("Scheduled tasks failed during model training.")
            return False
    else:
        logger.error("Scheduled tasks failed during DVC versioning.")
        return False
```

4. Model Serving & Monitoring

- ใช้ Flask API และ Docker ในการให้บริการโมเดล
- Monitor log และ latency ของ API เพื่อดูว่าระบบทำงานถูกต้องหรือไม่

```python
@app.route('/api/predict', methods=['POST'])
def api_predict():
    """API สำหรับพยากรณ์สายพันธุ์สุนัข"""
    if 'image' not in request.files:
        return jsonify({"error": "No file part in the request"}), 400

    file = request.files['image']
    if file.filename == '':
        return jsonify({"error": "No selected file"}), 400
```

```
(myenv) apisit72313@instance-20250311-051738:~/Mairu$ docker logs mairu-app-1
[2025-03-11 09:38:00 +0000] [1] [INFO] Starting gunicorn 20.1.0
[2025-03-11 09:38:00 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
[2025-03-11 09:38:00 +0000] [1] [INFO] Using worker: sync
[2025-03-11 09:38:00 +0000] [7] [INFO] Booting worker with pid: 7
2025-03-11 09:38:03,033 - apscheduler.scheduler - INFO - Adding job tentatively -- it will be properly scheduled
 when the scheduler starts
2025-03-11 09:38:03,034 - apscheduler.scheduler - INFO - Added job "scheduled_task" to job store "default"
2025-03-11 09:38:03,034 - apscheduler.scheduler - INFO - Scheduler started
2025-03-11 09:38:03,035 - app - INFO - Scheduler started. Will run every 3 days.
2025-03-11 09:38:03,040 - app.training - INFO - Forcing DVC versioning and model training...
2025-03-11 09:38:03,040 - app.training - INFO - Running scheduled DVC versioning and training job...
```

5. Feedback Loop

- เก็บ Feedback จากผู้ใช้งาน หากโมเดลทำนายผิด
- นำข้อมูลจาก feedback ไปเพิ่มในการฝึกโมเดลครั้งถัดไป

```python
@app.route('/api/feedback', methods=['POST'])
def api_feedback():
    """API สำหรับบันทึกข้อมูลการให้ฟีดแบ็ค"""
    session_id = request.cookies.get('session_id')
    user_label = request.form.get('user_label')
    filename = request.form.get('filename', f"dog_{int(time.time())}.jpg")

    if not session_id or session_id not in app.config['FEEDBACK_IMAGES']:
        return jsonify({"error": "No image available for feedback"}), 400

    if not user_label:
        return jsonify({"error": "No label provided"}), 400
```

```python
try:
    # Upload the image to temporary GCS location with the user-provided label
    file_bytes = app.config['FEEDBACK_IMAGES'][session_id]
    gcs_path = upload_to_gcs(file_bytes, user_label, filename, temp=True)
```

- ระบบบันทึก Feedback ของผู้ใช้ และอัปโหลดขึ้น GCS เพื่อนำไปปรับปรุงโมเดลในอนาคต

6. Version Control

- ใช้ Git + DVC สำหรับการเก็บเวอร์ชันของ Data, Code และ Model
- สามารถย้อนกลับไปใช้เวอร์ชันก่อนหน้าได้

```
PS E:\MLOps project\Mairu> git log --oneline
a11fce0 (HEAD -> veb, origin/veb) Merge branch 'veb' of https://gitlab.com/mtgu/Mairu into veb
34c37c9 edit schema initialize_db
877421b training data and better versioning
484f370 change versionin.py
26889a4 Chang Dockerfile
26d108c fix dvc[gs]
a101a32 add dvs-gs for versioning
1e46bc9 try to make dvc work
7f1b340 new path to model
44384fa make it work with dockerfile and compose
d0f9f06 docker file change for long run
c813aad run initialize_db when compose
854360b run initialize_db when compose
c6feaa0 sql for breed
```

☐	dog-data_v20250311_021721/	–	Folder	–
☐	dog-data_v20250311_021944/	–	Folder	–
☐	dog-data_v20250311_042537/	–	Folder	–
☐	dog-data_v20250311_042859/	–	Folder	–
☐	dog-data_v20250311_055320/	–	Folder	–
☐	dog-data_v20250311_065958/	–	Folder	–
☐	dog-data_v20250311_071542/	–	Folder	–
☐	dog-data_v20250311_072058/	–	Folder	–
☐	dog-data_v20250311_074746/	–	Folder	–
☐	dog-data_v20250311_075017/	–	Folder	–
☐	dog-data_v20250311_075340/	–	Folder	–
☐	dog-data_v20250311_080131/	–	Folder	–
☐	dog-data_v20250311_081336/	–	Folder	–
☐	dog-data_v20250311_092749/	–	Folder	–
☐	dog-data_v20250311_093706/	–	Folder	–
☐	dog-data_v20250311_093803/	–	Folder	–
☐	dvc-store/	–	Folder	–
☐	models/	–	Folder	–

7. Security and Privacy

- ใช้ Google IAM สำหรับควบคุมสิทธิ์การเข้าถึงข้อมูล
- กำหนด Service Account ในการเข้าถึงข้อมูล

8. Scalability

- ใช้ Cloud Run หรือ Kubernetes เพื่อรองรับผู้ใช้จำนวนมาก

```yaml
- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  entrypoint: gcloud
  args:
    - 'run'
    - 'deploy'
    - 'dog-classification'
    - '--image'
    - 'gcr.io/$PROJECT_ID/dog-classification:$COMMIT_SHA'
    - '--region'
    - 'us-central1'
    - '--platform'
    - 'managed'
    - '--allow-unauthenticated'
    - '--memory'
    - '1Gi'
```